

FACULDADE UNIFECAP

Algoritmos e Lógica de Programação

DESAFIO DE AUTOMAÇÃO DIGITAL: GESTÃO DE PEÇAS, QUALIDADE E ARMAZENAMENTO

Sistema em Python para controle de qualidade e armazenamento de peças

Aluno	Vinícius Franciscato
RA	254086
E-mail	vinifrantrafego@gmail.com
Curso	Inteligência Artificial e Automação Digital
Disciplina	Algoritmos e Lógica de Programação
Instituição	Faculdade UniFECAP
Data	Setembro de 2026

SUMÁRIO

1. Contextualização: por que automatizar a inspeção	3
2. Como estruturei o raciocínio lógico	3
2.1 O fluxo do sistema	3
2.2 Estruturas de dados: por que dicionário	4
2.3 Decisões: as condições de aprovação.....	5
2.4 Repetição: onde e por quê.....	6
2.5 As caixas: como o programa sabe que a caixa encheu	7
2.6 Funções	7
2.7 Validação de dados e tratamento de exceções	7
2.8 Remoção de peças	8
2.9 Recursos a mais: planilha e gravação dos dados.....	9
3. O programa em funcionamento: relatório final	10
4. Benefícios e desafios	11
Benefícios percebidos	11
Desafios enfrentados	12
5. Reflexão final: o protótipo num cenário real	12
6. Links	13
7. Referências	13

1. Contextualização: por que automatizar a inspeção

O desafio parte de uma situação comum na indústria: a inspeção das peças da linha de montagem é feita à mão. Alguém pesa, mede, olha a cor, decide se a peça passa e anota. Esse processo tem três problemas que o próprio enunciado aponta: atraso, falha de conferência e custo.

O atraso vem de a conferência ser uma etapa lenta no meio de uma linha que não para. A falha de conferência vem de a decisão depender da atenção de uma pessoa: no começo do turno ela confere com cuidado, no fim do turno o cansaço pesa, e o mesmo critério acaba aplicado de jeitos diferentes. Uma peça fora do padrão que passa chega ao cliente e volta como devolução. Uma peça boa descartada por engano é material e hora de máquina jogados fora. O custo é a soma disso tudo com o tempo gasto retrabalhando e corrigindo registros.

Automação na indústria serve principalmente para manter a qualidade num padrão e diminuir paradas (downtime), erros e retrabalho. O mesmo critério é aplicado em toda peça, do mesmo jeito, sem depender do cansaço de quem inspeciona. E com o registro automático dá para saber na hora quantas peças estão sendo reprovadas e por qual motivo, e corrigir a causa na máquina antes de perder um lote inteiro. Se de repente começam a aparecer muitas peças com comprimento fora do padrão, é sinal de que alguma máquina de corte saiu do ajuste, e isso só aparece rápido quando o registro é organizado.

Essa ideia de ligar as máquinas a sistemas que registram e analisam dados em tempo real é o que se chama de Indústria 4.0. O protótipo deste trabalho é um primeiro passo nessa direção: ainda depende de alguém informando os dados, mas a decisão, a organização em caixas e o relatório já são automáticos.

2. Como estruturei o raciocínio lógico

2.1 O fluxo do sistema

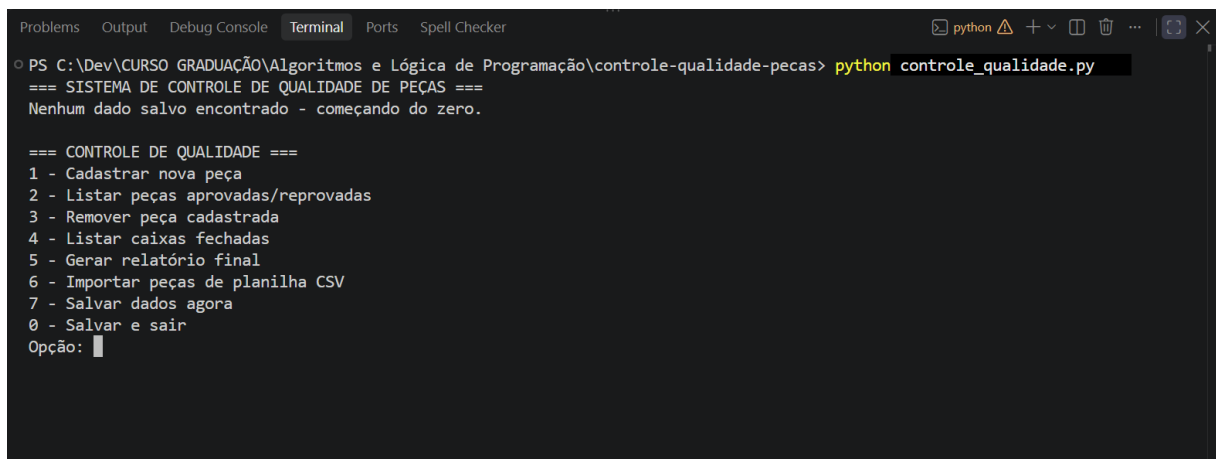
O programa é um menu que fica rodando até o operador escolher sair. A cada peça cadastrada o caminho é sempre o mesmo:

1. ler os dados (ID, peso, cor, comprimento), conferindo cada um;
2. mostrar o resumo e pedir confirmação;

3. avaliar a peça pelos três critérios;
4. se aprovada, guardar na caixa aberta; se reprovada, registrar os motivos;
5. se a caixa chegou a 10 peças, fechar.

O menu tem as cinco opções pedidas no enunciado e mais três que acrescentei: importar peças de uma planilha CSV, salvar os dados a qualquer momento e salvar ao sair.

Figura 1 – Abertura do programa, primeira execução, com o menu



```
PS C:\Dev\CURSO GRADUAÇÃO\Algoritmos e Lógica de Programação\controle-qualidade-pecas> python controle_qualidade.py
=== SISTEMA DE CONTROLE DE QUALIDADE DE PEÇAS ===
Nenhum dado salvo encontrado - começando do zero.

=== CONTROLE DE QUALIDADE ===
1 - Cadastrar nova peça
2 - Listar peças aprovadas/reprovadas
3 - Remover peça cadastrada
4 - Listar caixas fechadas
5 - Gerar relatório final
6 - Importar peças de planilha CSV
7 - Salvar dados agora
0 - Salvar e sair
Opção: █
```

Fonte: execução do programa pelo autor (2026).

2.2 Estruturas de dados: por que dicionário

Cada peça é um **dicionário**. Uma peça tem várias informações que andam juntas (ID, peso, cor, comprimento, se foi aprovada, os motivos e o número da caixa). Com variáveis soltas, eu teria que controlar cinco ou seis variáveis por peça, e não conseguiria guardar várias peças. Com o dicionário, cada peça é um "objeto" só, e as informações são acessadas pelo nome: `peca["peso"]`, `peca["cor"]`.

```
{
    "id": "P-101",
    "peso": 98.5,
    "cor": "azul",
    "comprimento": 15.0,
    "aprovada": True,
    "motivos": [],
    "numero_caixa": 1,
}
```

Todas as peças ficam numa **lista** (`pecas`). As caixas também são dicionários, numa segunda lista (`caixas`). Cada caixa guarda o número, a lista de IDs das peças que estão nela e se já foi fechada.

2.3 Decisões: as condições de aprovação

A regra de qualidade fica numa função só, `avaliar_pecas`. Ela confere os três critérios **um por um**, com três `if` independentes, e não com `if/elif`. Foi uma decisão consciente: com `elif`, a verificação pararia no primeiro problema encontrado. Uma peça com peso errado e cor errada apareceria só com "peso fora do padrão", e o relatório de motivos ficaria incompleto. Com três `if` separados, a peça guarda todos os motivos.

```
def avaliar_pecas(peso, cor, comprimento):
    """Devolve a lista de motivos de reprovação. Lista vazia significa peça
    aprovada."""
    motivos = []

    if esta_fora_da_faixa(peso, PESO_MINIMO, PESO_MAXIMO):
        motivos.append("peso fora do padrão")

    if cor not in CORES_ACEITAS:
        motivos.append("cor não aceita")

    if esta_fora_da_faixa(comprimento, COMPRIMENTO_MINIMO, COMPRIMENTO_MAXIMO):
        motivos.append("comprimento fora do padrão")

    return motivos
```

A peça está aprovada quando a lista de motivos volta vazia. Duas decisões ficaram registradas aqui:

- **Os limites contam como aprovados.** O enunciado diz "entre 95 g e 105 g", e interpretei que uma peça com exatamente 95 g ou 105 g está dentro do padrão.
- **Os critérios ficam em constantes** no começo do arquivo (`PESO_MINIMO`, `CAPACIDADE_CAIXA`, `CORES_ACEITAS`...). Se a empresa mudar o padrão, a alteração é feita num lugar só.

Figura 2 – Peça aprovada, digitada com vírgula e cor em maiúsculo

```
Opção: 1

--- CADASTRO DE PEÇA ---
ID da peça: v-01
Peso (g): 98,5
Cor: AZUL
Comprimento (cm): 15

Peça V-01 | Peso: 98.5 g | Cor: azul | Comprimento: 15.0 cm
Confirmar cadastro? (S/N): S
Peça APROVADA. Guardada na caixa 1 (1 de 10).
```

Fonte: execução do programa pelo autor (2026).

Figura 3 – Peça reprovada com dois motivos

```
Opção: 1

--- CADASTRO DE PEÇA ---
ID da peça: R-1
Peso (g): 120
Cor: preta
Comprimento (cm): 15

Peça R-1 | Peso: 120.0 g | Cor: preta | Comprimento: 15.0 cm
Confirmar cadastro? (S/N): S
Peça REPROVADA. Motivos:
  - peso fora do padrão
  - cor não aceita
```

Fonte: execução do programa pelo autor (2026).

2.4 Repetição: onde e por quê

- **while True no menu.** O programa precisa continuar aberto até o operador decidir sair; sem a repetição, ele executaria uma opção e terminaria. A saída acontece pelo break da opção 0.
- **while True na leitura de cada dado.** Se o operador digita um peso inválido, o programa avisa e pergunta de novo, até receber um valor aceitável. Assim nenhum dado errado chega a ser gravado.
- **for para percorrer listas.** Separar aprovadas de reprovadas, procurar uma peça pelo ID, contar os motivos de reprovação, contar as caixas fechadas e abertas.

2.5 As caixas: como o programa sabe que a caixa encheu

A função `guardar_na_caixa` procura a caixa que ainda está aberta. Se não houver nenhuma, cria uma nova. Depois coloca o ID da peça dentro e confere o tamanho da lista com `len`: quando chega a `CAPACIDADE_CAIXA` (10), a caixa é marcada como fechada. Na próxima peça aprovada não existe caixa aberta, então uma nova é criada.

```
caixa["ids_pecas"].append(id_peca)

if len(caixa["ids_pecas"]) == CAPACIDADE_CAIXA:
    caixa["fechada"] = True
```

Figura 4 – Opção 4: a caixa 1 fechada com 10 peças

Opção: 4

--- CAIXAS FECHADAS ---

Caixa 1 (10 peças): V-01 A-1 P-101 P-102 P-103 P-106 P-107 P-108 P-109 P-110

Fonte: execução do programa pelo autor (2026).

2.6 Funções

O código foi dividido em funções pequenas, cada uma com uma tarefa só, porque assim cada parte pode ser entendida e testada separadamente, e a mesma regra não aparece copiada em vários lugares. Organizei em dois tipos:

- **Funções que calculam e devolvem um resultado**, sem mostrar nada na tela: `avaliar_peca`, `converter_numero`, `separar_por_resultado`, `contar_motivos`, `contar_caixas`.
- **Funções que executam as opções do menu** e conversam com o operador: `cadastrar_peca`, `listar_pecas`, `remover_peca`, `listar_caixas_fechadas`, `gerar_relatorio`.

A vantagem ficou clara na importação da planilha: as peças do CSV passam pelas **mesmas** funções de verificação e avaliação do cadastro pelo teclado. A regra de qualidade existe num lugar só.

2.7 Validação de dados e tratamento de exceções

São duas coisas diferentes no programa:

- **Validação** é a regra que eu defini: peso precisa ser maior que zero, a cor não pode ter número, o ID não pode repetir. O programa confere e pede de novo.

- **Tratamento de exceção** (try/except) é para quando o Python não consegue executar algo: converter um texto que não é número, abrir um arquivo que não existe, usar uma biblioteca que não está instalada. Em vez de o programa quebrar com uma mensagem técnica, ele mostra um aviso e continua.

Um exemplo que mostra as duas juntas é a digitação do peso:

O operador digita	O que acontece
98,5	a vírgula é trocada por ponto e o valor é aceito como 98.5
abc	não é número: o programa avisa e pergunta de novo
9.8	é número válido, mas muito longe do padrão (perto de 100 g). Pode ser erro de digitação, então o programa pede confirmação. Não bloqueia, porque a peça pode realmente estar com defeito

O alerta de valor suspeito surgiu de uma pergunta minha no planejamento: se o valor está muito fora da realidade, o sistema deveria confirmar antes, porque é um erro que dá para perceber com antecedência.

Antes de gravar qualquer peça, o programa mostra o resumo e pede confirmação (S/N).

Figura 5 – Alerta de possível erro de digitação no peso 9.8

```
Opção: 1

--- CADASTRO DE PEÇA ---
ID da peça: A-1
Peso (g): 9.8
  Atenção: 9.8 g está muito fora do padrão (95 a 105 g).
  Pode ser erro de digitação.
  Confirma esse valor? (S/N): N
Peso (g): 98
Cor: AZUL
Comprimento (cm): 12

Peça A-1 | Peso: 98.0 g | Cor: azul | Comprimento: 12.0 cm
Confirmar cadastro? (S/N): S
Peça APROVADA. Guardada na caixa 1 (2 de 10).
```

Fonte: execução do programa pelo autor (2026).

2.8 Remoção de peças

Peças reprovadas e peças que estão numa caixa **aberta** podem ser removidas. Peças de uma caixa **fechada** não podem: a caixa já foi lacrada, e o registro no sistema precisa continuar igual ao que está dentro da caixa física. Se o

sistema deixasse remover, o relatório diria que a caixa tem 9 peças enquanto ela foi lacrada com 10.

Um detalhe: se a única peça de uma caixa aberta for removida, a caixa vazia também é apagada, para não ser contada como "caixa utilizada" no relatório.

Figura 6 – Tentativa de remover a peça P-101, que está na caixa fechada

```
Opção: 3

--- REMOÇÃO DE PEÇA ---
ID da peça a remover: P-101
A peça P-101 está na caixa 1, que já está fechada.
Peças de caixas fechadas não podem ser removidas pelo sistema.
```

Fonte: execução do programa pelo autor (2026).

2.9 Recursos a mais: planilha e gravação dos dados

Importação de planilha CSV (opção 6). Uma sugestão feita em aula foi receber as peças por planilha, e não só pelo teclado. Usei a biblioteca pandas para ler o arquivo. Cada linha passa pelas mesmas verificações do cadastro: linhas com erro são recusadas e listadas com o número da linha e o motivo. O pandas só é carregado quando essa opção é usada; se ele não estiver instalado, o programa avisa e as outras opções continuam funcionando.

Gravação em arquivo JSON. Sem isso, todas as peças se perdiam ao fechar o programa. Agora os dados são salvos no arquivo dados_pecas.json ao sair (ou pela opção 7) e carregados de novo ao abrir. Na primeira execução, quando o arquivo ainda não existe, o programa avisa que está começando do zero.

Figura 7 – Importação da planilha de exemplo

```
Opção: 6

--- IMPORTAÇÃO DE PLANILHA CSV ---
Nome do arquivo CSV (ex.: pecas_exemplo.csv): pecas_exemplo.csv

Linhas lidas: 14
Peças importadas: 14 (aprovadas: 11 - reprovadas: 3)
Linhas recusadas: 0
```

Fonte: execução do programa pelo autor (2026).

Figura 8 – Programa aberto de novo, carregando os dados salvos

```
Opção: 0
17 peça(s) e 2 caixa(s) salvas em dados_pecas.json.
Sistema encerrado.
❖ PS C:\Dev\CURSO GRADUAÇÃO\Algoritmos e Lógica de Programação\controle-qualidade-pecas> python controle_qualidade.py
=== SISTEMA DE CONTROLE DE QUALIDADE DE PEÇAS ===
Dados carregados de dados_pecas.json: 17 peça(s) e 2 caixa(s).
```

Fonte: execução do programa pelo autor (2026).

3. O programa em funcionamento: relatório final

O relatório consolidado mostra o total de peças aprovadas e reprovadas, quantas vezes cada motivo de reprovação apareceu, quais peças foram reprovadas e por quê, e as caixas utilizadas, separadas em fechadas e em aberto.

Para os prints, cadastrei três peças pelo teclado (V-01 e A-1 aprovadas, R-1 reprovada) e importei a planilha de exemplo, com 14 peças. A opção 2 mostra as peças separadas em aprovadas e reprovadas, cada aprovada com o número da caixa e cada reprovada com os seus motivos.

Figura 9 – Opção 2: peças aprovadas e reprovadas

```
Opção: 2

--- PEÇAS APROVADAS (13) ---
V-01 | 98.5 g | azul | 15.0 cm | caixa 1
A-1 | 98.0 g | azul | 12.0 cm | caixa 1
P-101 | 98.5 g | azul | 15.0 cm | caixa 1
P-102 | 101.2 g | verde | 12.5 cm | caixa 1
P-103 | 100.0 g | azul | 18.0 cm | caixa 1
P-106 | 96.0 g | verde | 11.0 cm | caixa 1
P-107 | 103.0 g | verde | 19.0 cm | caixa 1
P-108 | 97.5 g | azul | 14.0 cm | caixa 1
P-109 | 104.0 g | verde | 16.0 cm | caixa 1
P-110 | 100.0 g | azul | 10.0 cm | caixa 1
P-111 | 95.0 g | verde | 20.0 cm | caixa 2
P-113 | 100.5 g | verde | 13.0 cm | caixa 2
P-114 | 98.0 g | azul | 17.0 cm | caixa 2

--- PEÇAS REPROVADAS (4) ---
R-1 | 120.0 g | preta | 15.0 cm
- peso fora do padrão
- cor não aceita
P-104 | 112.0 g | azul | 15.0 cm
- peso fora do padrão
P-105 | 99.0 g | vermelha | 22.0 cm
- cor não aceita
- comprimento fora do padrão
P-112 | 88.0 g | preta | 8.0 cm
- peso fora do padrão
- cor não aceita
- comprimento fora do padrão
```

Fonte: execução do programa pelo autor (2026).

Figura 10 – Relatório final

```
Opção: 5

=== RELATÓRIO FINAL ===
Peças cadastradas: 17
Peças aprovadas: 13
Peças reprovadas: 4

Reprovações por motivo:
- peso fora do padrão: 3
- cor não aceita: 3
- comprimento fora do padrão: 2
(uma peça pode ter mais de um motivo)

Peças reprovadas:
R-1
- peso fora do padrão
- cor não aceita
P-104
- peso fora do padrão
P-105
- cor não aceita
- comprimento fora do padrão
P-112
- peso fora do padrão
- cor não aceita
- comprimento fora do padrão

Caixas utilizadas: 2 - Caixas Fechadas: 1 - Caixas em aberto: 1
```

Fonte: execução do programa pelo autor (2026).

O resultado foi: 17 peças cadastradas, 13 aprovadas e 4 reprovadas. As duas primeiras aprovadas e as oito primeiras aprovadas da planilha encheram a caixa 1, que fechou; as outras três foram para a caixa 2, que ficou em aberto. Por isso o relatório mostra 2 caixas utilizadas, 1 fechada e 1 em aberto.

4. Benefícios e desafios

Benefícios percebidos

- **Critério igual para toda peça.** A decisão não depende de quem está inspecionando nem da hora do turno.
- **Motivo registrado.** O relatório mostra quantas peças caíram em cada critério, o que ajuda a achar a causa do problema na linha.
- **Menos erro de digitação chegando ao registro.** Valores impossíveis são recusados e valores suspeitos pedem confirmação.
- **Caixas organizadas sem conta manual.** O sistema sabe em que caixa está cada peça e fecha a caixa na hora certa.
- **Os dados não se perdem** ao fechar o programa.

Desafios enfrentados

O mais difícil foi entender como as partes se ligam: separar o que é rodar o programa do que é editar o código, e entender regras que dependem do mundo real, como não poder remover peça de uma caixa já lacrada. Depois que entendi a história da linha de montagem, o código fez sentido. É bem legal de aprender.

Durante a revisão e os testes, apareceram problemas que precisaram de correção:

- **Peso "nan" aprovava a peça.** O Python aceita o texto nan como número ("não é um número"), e toda comparação com ele dá falso, então a peça não caía em nenhum limite e saía aprovada. Nenhum dos testes planejados pegou; foi achado revisando o código. A correção foi conferir, antes de converter, se o texto tem só dígitos, ponto e sinal de menos.
- **Caixa vazia contando no relatório.** Ao remover a única peça de uma caixa aberta, o relatório continuava dizendo "Caixas em aberto: 1". A correção foi apagar a caixa aberta que fica vazia.
- **Arquivo de dados corrompido.** Se o `dados_pecas.json` estivesse com defeito, o programa começava do zero e, ao salvar, apagava o conteúdo antigo sem avisar. Agora ele avisa logo na abertura.
- **Célula vazia na planilha.** O pandas lê célula vazia como "nan". A solução foi ler tudo como texto e passar pelas mesmas verificações do teclado.

O programa foi testado em 23 cenários, entre eles: valores nos limites (95 g, 105 g, 10 cm, 20 cm), todos os tipos de entrada inválida, caixa completa, remoção bloqueada, planilha com erros, planilha sem uma coluna, arquivo inexistente e arquivo de dados corrompido.

5. Reflexão final: o protótipo num cenário real

O ponto que mudaria é a entrada: em vez do operador digitar no `input()`, o peso viria da balança e a cor do sensor. A avaliação e as caixas continuariam iguais. A validação continuaria necessária, porque sensor também falha, mas a confirmação S/N teria que virar uma separação da peça para conferência. Com volume maior, trocaria o arquivo JSON por um banco de dados e faria uma tela para quem supervisiona a linha.

Com inteligência artificial, uma câmera poderia substituir o sensor de cor e ir além dele, identificando defeitos visuais como riscos e rebarbas, que os três critérios de hoje não pegam. E como o sistema já registra os motivos de reprovação, esse histórico poderia ser usado para perceber quando uma máquina está começando a sair do ajuste, antes de as peças começarem a ser reprovadas.

Na integração industrial, o fechamento da caixa poderia disparar a etiqueta e avisar o setor de expedição, e o relatório poderia ficar num painel atualizado durante o turno, em vez de ser gerado só quando alguém pede.

6. Links

- **Código-fonte (GitHub):** <https://github.com/vinifranciscato/controle-qualidade-pecas>
- **Vídeo de apresentação:** <https://youtu.be/lqdcl4ug2rE>

7. Referências

- Material das aulas ao vivo da disciplina Algoritmos e Lógica de Programação, UniFECAF, 2026.
- PYTHON SOFTWARE FOUNDATION. *Documentação do Python 3*. Disponível em: <https://docs.python.org/3/>.
- PYTHON BRASIL. Disponível em: <https://python.org.br/>.
- PANDAS. *pandas documentation*. Disponível em: <https://pandas.pydata.org/docs/>.